

PATRÓN DE DISEÑO: COMANDO (COMMAND)

Capítulo 30 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

En general, una aplicación orientada a objetos consiste en un conjunto de objetos que interactúan entre sí, cada uno ofreciendo funcionalidad limitada y enfocada. En respuesta a la interacción del usuario, la aplicación realiza algún tipo de procesamiento. Para ello, la aplicación hace uso de los servicios de diferentes objetos para el procesamiento requerido. En términos de implementación, la aplicación puede depender de un objeto designado que invoca métodos sobre estos objetos pasando los datos y argumentos requeridos. Este objeto designado puede denominarse invocador (invoker). El conjunto de objetos que contienen la implementación real de los servicios requeridos para el procesamiento de la solicitud puede denominarse objetos Receptor (Receiver).

En este diseño, la aplicación que reenvía la solicitud y los objetos Receptor que ofrecen los servicios necesarios para procesarla están estrechamente vinculados entre sí, ya que interactúan directamente. Esto podría resultar en un conjunto de sentencias condicionales if en la implementación del invocador:

```
if (TipoSolicitud == TipoA) {  
    // hacer algo  
}  
if (TipoSolicitud == TipoB) {  
    // hacer algo  
}
```

Cuando se agrega un nuevo tipo de funcionalidad a la aplicación, el código existente necesita ser modificado, lo que viola el principio abierto-cerrado del diseño orientado a objetos:

- Abierto para la extensión — Debe ser posible alterar el comportamiento de un módulo o agregar nuevas características a su funcionalidad.
- Cerrado para la modificación — Dicho módulo no debe permitir que su código sea modificado.

Usando el Patrón Comando, el invocador que emite una solicitud y el conjunto de objetos Receptor que prestan servicios pueden desacoplarse. El Patrón Comando sugiere crear una

abstracción para el procesamiento que se llevará a cabo o la acción que se tomará en respuesta a las solicitudes del cliente.

Esta abstracción puede diseñarse para declarar una interfaz común implementada por diferentes implementadores concretos denominados objetos Comando. Cada objeto Comando representa un tipo diferente de solicitud del cliente y el procesamiento correspondiente. El objeto Comando es responsable de ofrecer la funcionalidad necesaria para procesar la solicitud que representa, pero no contiene la implementación real de dicha funcionalidad. Los objetos Comando hacen uso de objetos Receptor para ofrecer esta funcionalidad.

2. Flujo General de Aplicación

Cuando la aplicación cliente necesita ofrecer un servicio en respuesta a una interacción del usuario:

1. Crea los objetos Receptor necesarios.
2. Crea un objeto Comando apropiado y lo configura con los objetos Receptor creados en el Paso 1.
3. Crea una instancia del invocador y la configura con el objeto Comando creado en el Paso 2.
4. El invocador invoca el método `execute()` sobre el objeto Comando.
5. Como parte de su implementación del método `execute`, un objeto Comando típico invoca los métodos necesarios sobre los objetos Receptor para proveer el servicio requerido a su llamador.

En el nuevo diseño:

- El cliente/invocador no interactúa directamente con los objetos Receptor; por lo tanto, están completamente desacoplados entre sí.
- Cuando la aplicación necesita ofrecer una nueva funcionalidad, se puede agregar un nuevo objeto Comando sin requerir cambios en el código existente del invocador. De este modo el nuevo diseño se adhiere al principio abierto-cerrado.
- Dado que la solicitud se diseña en forma de objeto, esto abre un conjunto completamente nuevo de posibilidades, tales como:
 - Almacenar un objeto Comando en medios persistentes para ejecutarlo posteriormente.
 - Aplicar procesamiento inverso para soportar la funcionalidad de deshacer (undo).
 - Agrupar diferentes objetos Comando para ejecutarlos como una unidad.

3. Ejemplo I — Cliente FTP

3.1 Diseño Inicial sin el Patrón Comando

Se construirá una aplicación que simula el funcionamiento de un cliente FTP. Una interfaz de usuario FTP simple puede diseñarse usando dos objetos `JList` para mostrar los sistemas de

archivos local y remoto, y cuatro objetos JButton para iniciar diferentes tipos de solicitudes: subir (upload), descargar (download), eliminar (delete) y salir (exit).

Cuando cada uno de los objetos JButton es creado, se le asigna una instancia de la clase ButtonHandler que implementa la interfaz integrada ActionListener. Esto significa que cada vez que se hace clic sobre un JButton, se ejecuta el método actionPerformed del ButtonHandler.

Dado que la misma instancia del ButtonHandler se establece como ActionListener para todos los JButton de la interfaz, el método actionPerformed se llama para todos ellos. Por lo tanto, el objeto ButtonHandler debe verificar qué botón fue presionado y llevar a cabo el procesamiento apropiado. El código en el método actionPerformed resulta poco elegante con un conjunto de sentencias condicionales:

Listado 3.1 — Clase ButtonHandler (Diseño Inicial)

```
class ButtonHandler implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        if (e.getActionCommand().equals(FTPGUI.EXIT)) {
            System.exit(1);
        }
        if (e.getActionCommand().equals(FTPGUI.UPLOAD)) {
            int index = localList.getSelectedIndex();
            String selectedItem =
                localList.getSelectedValue().toString();
            ((DefaultListModel) localList.getModel()).remove(index);
            ((DefaultListModel) remoteList.getModel())
                .addElement(selectedItem);
        }
        if (e.getActionCommand().equals(FTPGUI.DOWNLOAD)) {
            int index = remoteList.getSelectedIndex();
            String selectedItem =
                remoteList.getSelectedValue().toString();
            ((DefaultListModel) remoteList.getModel()).remove(index);
            ((DefaultListModel) localList.getModel())
                .addElement(selectedItem);
        }
        if (e.getActionCommand().equals(FTPGUI.DELETE)) {
            int index = localList.getSelectedIndex();
            if (index >= 0)
                ((DefaultListModel) localList.getModel()).remove(index);
            index = remoteList.getSelectedIndex();
            if (index >= 0)
                ((DefaultListModel) remoteList.getModel()).remove(index);
        }
    }
}
```

A medida que se agregan más botones y elementos de menú al UI del FTP, el código puede volverse rápidamente desordenado. Además, cuando se agrega un nuevo botón, el código

existente del método `actionPerformed` necesita modificarse, violando el principio abierto-cerrado.

3.2 Rediseño con el Patrón Comando

Aplicando el Patrón Comando, se define una abstracción en forma de interfaz `CommandInterface` para la funcionalidad asociada con los diferentes botones del UI del cliente FTP:

```
interface CommandInterface {  
    public void processEvent();  
}
```

Se diseña un conjunto de nuevas clases de botones, cada una correspondiente a un tipo diferente de solicitud, como subclases de la clase `JButton`. Estas subclases implementan la interfaz `CommandInterface`. Como parte de su implementación del método `processEvent`, cada subclase ofrece la funcionalidad requerida para procesar la solicitud que representa.

Listado 3.2 — Subclases de JButton como Objetos Comando

```
class UploadButton extends JButton implements CommandInterface {  
    public UploadButton(String name) { super(name); }  
    public void processEvent() {  
        int index = localList.getSelectedIndex();  
        String selectedItem =  
            localList.getSelectedValue().toString();  
        ((DefaultListModel)localList.getModel()).remove(index);  
        ((DefaultListModel)remoteList.getModel()).addElement(selectedItem);  
    }  
}  
  
class DownloadButton extends JButton implements CommandInterface {  
    public DownloadButton(String name) { super(name); }  
    public void processEvent() {  
        int index = remoteList.getSelectedIndex();  
        String selectedItem =  
            remoteList.getSelectedValue().toString();  
        ((DefaultListModel)remoteList.getModel()).remove(index);  
        ((DefaultListModel)localList.getModel()).addElement(selectedItem);  
    }  
}  
  
class DeleteButton extends JButton implements CommandInterface {  
    public DeleteButton(String name) { super(name); }  
    public void processEvent() {  
        int index = localList.getSelectedIndex();  
        if (index >= 0)  
            ((DefaultListModel)localList.getModel()).remove(index);  
        index = remoteList.getSelectedIndex();  
        if (index >= 0)
```

```

        ((DefaultListModel)remoteList.getModel()).remove(index);
    }
}

class ExitButton extends JButton implements CommandInterface {
    public ExitButton(String name) { super(name); }
    public void processEvent() {
        System.exit(1);
    }
}

```

Con este nuevo diseño, el método `actionPerformed` queda altamente simplificado a apenas dos líneas de código:

```

class ButtonHandler implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        CommandInterface CommandObj =
            (CommandInterface) e.getSource();
        CommandObj.processEvent();
    }
}

```

En el nuevo diseño, cuando se agrega un nuevo botón o elemento de menú, solo es necesario crear un nuevo objeto Comando como implementador de `CommandInterface`. El nuevo objeto Comando puede agregarse a la aplicación de manera transparente, sin requerir cambios en el código existente del método `actionPerformed`. Como contrapartida, esto resulta en un mayor número de clases.

4. Ejemplo II — Gestión de Biblioteca

4.1 Descripción del Problema

Se construirá una aplicación para gestionar ítems en una base de datos de ítems de biblioteca. Los ítems de la biblioteca incluyen libros, CDs, videos y DVDs. Estos ítems se organizan en categorías y un ítem determinado puede pertenecer a una o más categorías. Por ejemplo, un nuevo video de película puede pertenecer tanto a la categoría Video como a la categoría Nuevos Lanzamientos.

Para simplificar, la aplicación gestionará únicamente las operaciones de agregar y eliminar ítems. Aplicando el Patrón Comando, la acción a realizar para procesar las solicitudes de agregar y eliminar ítems se diseña como implementadores de una interfaz `CommandInterface` común.

4.2 Clases Item y Category

Se definen dos clases — `Item` y `Category` — para representar un ítem típico de biblioteca y una categoría de ítems, respectivamente. Un objeto `Category` mantiene una lista de sus ítems miembros. De manera similar, un objeto `Item` mantiene la lista de todas las categorías de las que forma parte.

Listado 4.1 — Clases `Item` y `Category`

```
public class Item {
    private HashMap categories;
    private String desc;

    public Item(String s) {
        desc = s;
        categories = new HashMap();
    }
    public String getDesc() { return desc; }
    public void add(Category cat) {
        categories.put(cat.getDesc(), cat);
    }
    public void delete(Category cat) {
        categories.remove(cat.getDesc());
    }
}

public class Category {
    private HashMap items;
    private String desc;

    public Category(String s) {
        desc = s;
        items = new HashMap();
    }
    public String getDesc() { return desc; }
    public void add(Item i) {
        items.put(i.getDesc(), i);
        System.out.println("Item '" + i.getDesc()
            + "' ha sido agregado a la Categoría '"
            + getDesc() + "'");
    }
    public void delete(Item i) {
        items.remove(i.getDesc());
        System.out.println("Item '" + i.getDesc()
            + "' ha sido eliminado de la Categoría '"
            + getDesc() + "'");
    }
}
```

4.3 Invocador `ItemManager`

Se define una clase invocador `ItemManager` que contiene un objeto `Comando`, invoca el método `execute` del objeto `Comando` como parte de la implementación de su método `process`, y

proporciona un método `setCommand` para permitir que los objetos cliente lo configuren con un objeto Comando.

```
public class ItemManager {
    CommandInterface command;

    public void setCommand(CommandInterface c) {
        command = c;
    }
    public void process() {
        command.execute();
    }
}
```

4.4 Flujo de la Aplicación — Cliente CommandTest

Para agregar o eliminar un ítem, el cliente `CommandTest` realiza los siguientes pasos:

6. Crea los objetos `Item` y `Category` necesarios. Estos objetos son los Receptores.
7. Crea un objeto Comando apropiado que corresponda a la solicitud. Los objetos Receptor creados en el Paso 1 se pasan al objeto Comando en el momento de su creación.
8. Crea una instancia del `ItemManager` y lo configura con el objeto Comando creado en el Paso 2.
9. Invoca el método `process()` del `ItemManager`. El `ItemManager` invoca el método `execute` sobre el objeto Comando. El objeto Comando a su vez invoca los métodos necesarios del objeto Receptor. Los objetos Receptor `Item` y `Category` realizan el procesamiento real de la solicitud.

Listado 4.2 — Clase Cliente CommandTest

```
public class CommandTest {
    public static void main(String[] args) {
        // Agregar un item a la categoria CD
        Item CD = new Item("A Beautiful Mind");
        Category catCD = new Category("CD");
        CommandInterface command = new AddCommand(CD, catCD);
        ItemManager manager = new ItemManager();
        manager.setCommand(command);
        manager.process();

        // Agregar otro item a la categoria CD
        CD = new Item("Duet");
        catCD = new Category("CD");
        command = new AddCommand(CD, catCD);
        manager.setCommand(command);
        manager.process();

        // Agregar un item a la categoria Nuevos Lanzamientos
        CD = new Item("Duet");
```

```

        catCD = new Category("Nuevos Lanzamientos");
        command = new AddCommand(CD, catCD);
        manager.setCommand(command);
        manager.process();

        // Eliminar el item de la categoria Nuevos Lanzamientos
        command = new DeleteCommand(CD, catCD);
        manager.setCommand(command);
        manager.process();
    }
}

```

Cuando el programa cliente se ejecuta, se muestra la siguiente salida:

```

Item 'A Beautiful Mind' ha sido agregado a la Categoria 'CD'
Item 'Duet' ha sido agregado a la Categoria 'CD'
Item 'Duet' ha sido agregado a la Categoria 'Nuevos Lanzamientos'
Item 'Duet' ha sido eliminado de la Categoria 'Nuevos Lanzamientos'

```

5. Comparación: Diseño Inicial versus Diseño con Patrón Comando

Tabla 1 — Diseño sin Patrón Comando versus con Patrón Comando

Aspecto	Sin Patrón Comando	Con Patrón Comando
Acoplamiento	El invocador y los Receptores están fuertemente acoplados mediante interacción directa.	El invocador y los Receptores están completamente desacoplados a través del objeto Comando.
Extensibilidad	Agregar un nuevo tipo de solicitud requiere modificar el código existente del invocador (if adicionales).	Agregar una nueva funcionalidad solo requiere crear un nuevo objeto Comando, sin modificar el invocador.
Principio abierto-cerrado	Violado: el código del invocador debe modificarse con cada nueva funcionalidad.	Respetado: el invocador no necesita cambiar cuando se agregan nuevos Comandos.
Funcionalidades adicionales	Difícil de implementar undo, logging de acciones o agrupación de comandos.	Fácilmente extensible para soportar undo, persistencia de comandos y ejecución agrupada.
Cantidad de clases	Menor número de clases.	Mayor número de clases (una por tipo de comando), pero con mejor mantenibilidad.

6. Preguntas de Práctica

10. En el Ejemplo I, diferentes clases Comando concretas están diseñadas como clases internas (inner classes). Rediseñe e implemente la aplicación de ejemplo con las diferentes clases Comando como clases externas.
11. Agregue un nuevo método `undo()` a la interfaz `CommandInterface` en los Ejemplos I y II. Mejore las diferentes clases Comando que implementan esta interfaz para ofrecer la funcionalidad necesaria para deshacer el efecto del método `execute` correspondiente.
12. Mejore la aplicación del Ejemplo II para incluir la capacidad de registrar la fecha y hora en que se realiza una operación de agregar o eliminar específica.
13. Mejore la aplicación del Ejemplo II para agregar la funcionalidad de mover, que permite mover un ítem de una categoría a otra. Implemente la funcionalidad de mover como una combinación de eliminar seguida de agregar. Ambas operaciones (eliminar y agregar) deben ejecutarse juntas para proveer la funcionalidad de mover.